

VoteMelhor — Analisando os dados do Senado

Acompanhe a análise no código. Clone o repositório para explorar a implementação:

<https://github.com/10xdev-startup/vote-melhor>

```
git clone https://github.com/10xdev-startup/vote-melhor.git
```

Fontes, usos e caminho técnico

Este dossiê documenta **como a VoteMelhor consulta e interpreta as fontes oficiais do Senado**: quais informações estão disponíveis, para que servem e como chegam à aplicação. O percurso passa pela escolha da fonte, pelo acesso aos arquivos ou à API, pela interpretação dos campos e pela apresentação com referências verificáveis.

Começamos pelas leituras que esses dados permitem e pelos cuidados aprendidos na integração. Depois mostramos os arquivos do código, o fluxo técnico e o tratamento dos problemas de conexão. O objetivo é tornar esse caminho compreensível e reproduzível.

1. Podemos acompanhar uma decisão pelo assunto ou pelo senador

Nas votações nominais, a fonte permite associar parlamentar, escolha de voto, data e matéria. Isso abre dois caminhos: “como este senador votou?” e “quem votou a favor ou contra este assunto?”. A Vote Melhor já organiza o mesmo conjunto nessas duas leituras.

Um aprendizado importante: **uma matéria pode ter várias votações**. Texto-base e destaques são decisões diferentes; ver resultados aprovados e rejeitados para a mesma proposição não é, por si só, uma contradição. Agrupar essas etapas ajuda a entender o que realmente foi decidido, em vez de reduzir tudo a um único “sim” ou “não”. Veja a implementação [por assunto](#) e [por senador](#).

2. Nem toda linha da lista de votos é um voto

Um cuidado necessário na integração é distinguir voto e comparecimento. O campo de voto também traz situações de comparecimento, licenças, impedimentos e registros de votação secreta. P-NRV, por exemplo, significa **presente, mas não registrou voto**. Tratar esse registro como falta produziria uma conclusão errada sobre o parlamentar.

No levantamento interno de 16/08/2026, cerca de **38% dos 288.774 registros** da série consultada de 1991 a 2026 identificavam uma escolha de voto. Esse número é histórico; não é uma contagem atualizada deste dossiê. A descoberta mostra por que contar linhas sem interpretar os códigos distorce a informação. Em votação secreta, o registro Votou não revela a escolha individual. Fontes: [levantamento da API](#) e [classificação utilizada no código](#).

3. O orçamento permite comparar autorização e execução

O conjunto de despesas traz dotação inicial e atualizada, além de valores empenhados, liquidados e pagos, desde 2013. É útil para perguntar **quanto foi autorizado e quanto aparece como pago em uma ação orçamentária**, respeitando o exercício consultado. Esses campos representam etapas diferentes; não devem ser somados como despesas independentes. A fonte descreve o conjunto no [portal de orçamento do Senado](#).

4. Também dá para comparar receita prevista com arrecadada

O Senado publica receitas próprias desde 2012, com previsão e arrecadação. Isso permite explorar diferenças entre o que se esperava receber e o que foi registrado, por natureza de receita e período. **Receitas próprias não são sinônimo de todo o financiamento da instituição**. Há arquivos CSV e JSON: a mesma informação pode atender tanto quem usa planilhas quanto quem desenvolve aplicações. Fontes: [conjunto oficial de receitas próprias](#) e [catálogo integrado](#).

Por onde começar a explorar

Pergunta	Onde começar	Cuidado na leitura
Como um senador votou em um tema?	Retrospecto do senador	Abrir a votação e conferir qual etapa da matéria foi decidida
Quem apoiou uma proposta?	Votações agrupadas por matéria	Separar texto-base, destaques e votações secretas
O autorizado virou pagamento?	Dotação e despesas executadas	Comparar a mesma ação e o mesmo exercício
A receita atingiu a previsão?	Receitas próprias	Comparar natureza e período equivalentes

As duas primeiras leituras já são organizadas pela aplicação. As perguntas financeiras indicam explorações possíveis nos arquivos: este dossiê não afirma que já exista um painel pronto para todas elas, nem apresenta diferenças de valores como irregularidades.

Como tornamos esses dados consultáveis

Para chegar a essas informações, integramos arquivos para download e consultas a uma API. Durante esse trabalho, encontramos uma dificuldade antes mesmo da leitura dos dados: a conexão segura com alguns endereços não terminava de ser negociada. A seguir estão o percurso real da integração, a solução adotada e os limites do que conseguimos comprovar.

Duas portas de entrada

O [portal de dados abertos do Senado, no grupo Orçamento](#), reúne conjuntos financeiros e contábeis com arquivos para download. O catálogo da Vote Melhor atribui nomes e períodos a esses arquivos e conserva seus endereços oficiais. Um exemplo usado na investigação é o [CSV de receitas do Senado](#).

Para dados legislativos, o código utiliza a [API de dados abertos do Senado](#). São consultas distintas: senadores em exercício, votações e tramitação de matérias. Ter acesso a um desses caminhos não significa que todos os outros estejam funcionando.

Caminho	O que chega	Onde a pessoa consulta
Arquivos financeiros	CSV e outros formatos do catálogo	Fonte de dados
Dados legislativos	Respostas JSON por consulta	Tela Senado e jornadas legislativas

A referência para essa separação está no [catálogo curado](#) e no [cliente da API legislativa](#). As afirmações sobre implementação neste documento correspondem a esse commit, não às mudanças locais que adicionaram a biblioteca de dossiês.

O percurso até a tela

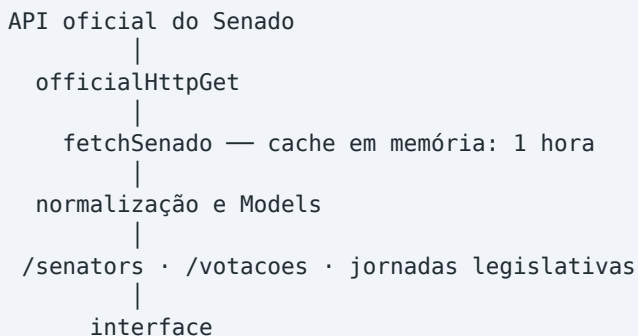
O navegador pede uma amostra à API da Vote Melhor. O backend escolhe o endereço a partir do catálogo, baixa o arquivo e interpreta suas linhas. A requisição da pessoa contém o identificador do arquivo; ela não fornece um endereço arbitrário para o servidor acessar.

ARQUIVOS FINANCEIROS

```
Pessoa → Fonte de dados → API Vote Melhor
      |
      | catálogo curado
      |
      | fetchSourceFile
      |
      | officialHttpGet
      |
      | arquivo oficial
      |
      | parseSpreadsheet
      |
      | amostra na tela
```

Esse caminho não grava o arquivo financeiro em um banco. A amostra é produzida a partir da resposta obtida da origem. Se o download falhar, a tela precisa comunicar o erro, não inventar uma amostra. Veja o [controller do catálogo](#) e o [download de arquivos](#).

DADOS LEGISLATIVOS



No cliente legislativo, o cache em memória evita repetir imediatamente as mesmas consultas. Ele expira após uma hora e não constitui um acervo permanente. A consulta de votações percorre os anos desde 2019 até o ano corrente do processo. Isso descreve o recorte da aplicação; não afirma que a API oficial tenha dados somente a partir de 2019. Evidência: [fetchSenado](#).

Quando uma conexão parecia estar fora do ar

A investigação interna registrou falhas do `fetch` do Node com o erro `UND_ERR_CONNECT_TIMEOUT`, por volta de 10,5 segundos. Na mesma investigação, `curl` conseguia baixar o arquivo em aproximadamente 0,4 segundo. O registro relata reprodução em WSL2 e em um container Linux, ambos com Node 22.20.0.

Essas são **medições históricas**, registradas na [investigação de TLS](#). Não são testes de disponibilidade executados para este dossiê.

A conexão TCP podia ser estabelecida, mas a negociação TLS não terminava com determinadas configurações do Node. TLS é a negociação que prepara a conexão HTTPS. Por isso, “timeout de conexão” não significava necessariamente um servidor desligado ou um arquivo inexistente.

Evidência histórica	O que permite concluir
curl recebeu o arquivo	A origem respondia àquela ferramenta naquele momento
TCP conectava e TLS travava	A falha observada estava depois da abertura do socket
Limitar TLS a 1.2 funcionou	Havia um contorno reproduzível naquele ambiente
Mais de um ambiente reproduziu	O sintoma não ficou restrito a uma única execução

O registro não isolou qual característica do anúncio inicial do cliente, o *ClientHello*, provocava o comportamento. Portanto, não podemos concluir que todo o Senado seja incompatível com TLS 1.3, nem apontar um equipamento específico como causa. A documentação histórica contém hipóteses mais fortes; aqui a conclusão fica limitada ao que as observações sustentam.

A solução que está no código

O utilitário [officialHttpGet](#) centraliza as chamadas HTTPS às fontes oficiais. Usa `node:https` e fixa a versão máxima de TLS em 1.2 para três hosts explicitamente listados:

- `www.senado.gov.br`
- `www12.senado.leg.br`
- `legis.senado.leg.br`

Os demais hosts seguem a configuração normal do Node. O código não desliga a validação dos certificados. Isso importa: conseguir baixar um arquivo não justificaria aceitar qualquer certificado apresentado pela rede.

O mesmo utilitário segue redirecionamentos, com limite de cinco saltos. Um endereço da API pode responder indicando outro endereço; ignorar essa resposta faria o consumidor tentar interpretar um corpo vazio. O transporte resolve endereços relativos e absolutos e retorna status, cabeçalhos e corpo para o consumidor decidir como tratar a resposta.

A [mudança que centralizou o transporte](#) permite conferir a passagem dos clientes anteriores para esse utilitário. Ela é uma evidência de implementação, não uma prova da causa do problema externo.

Como conferir e contribuir

Comece pelo endereço oficial e identifique qual dos dois caminhos está falhando. Depois acompanhe o identificador do arquivo ou os parâmetros da consulta até o transporte. Os pontos de entrada principais são:

Responsabilidade	Arquivo no backend
Endereços e nomes dos conjuntos	<code>DataCatalogModel.ts</code>
Download de arquivos	<code>fetchSourceFile.ts</code>
Amostra e filtros do catálogo	<code>DataCatalogController.ts</code>
HTTPS, TLS e redirects	<code>officialHttpGet.ts</code>
Consultas legislativas e cache	<code>fetchSenado.ts</code>
Padronização de votos	<code>normalizeSenadoVote.ts</code>

As [regras de normalização de votos](#) e os [testes de download](#) ajudam a separar transformação de dados e falha de rede.

Para reproduzir os testes unitários pertinentes, na raiz do repositório:

```
npm test -w backend -- --runInBand \
  src/tests/fetchSourceFile.test.ts \
  src/tests/fetchSenadoProcess.test.ts \
  src/tests/fetchSenado.test.ts
```

Na revisão que preparou este piloto, os 12 testes dessas três suítes passaram. Eles usam respostas HTTPS simuladas: conferem o comportamento do código nos cenários descritos, mas não comprovam que o handshake com o Senado funcione hoje.

Se for necessário repetir a investigação de rede, registre data, versão do Node, ambiente, URL, status e duração. Compare a negociação padrão com o limite TLS por host. Não amplie a exceção para outros domínios por semelhança do nome.

Limites desta leitura

- Este é um retrato do código no SHA indicado no cabeçalho. Uma publicação posterior pode atualizar o texto; o piloto de dossiês não guarda histórico.
- A data de publicação deste documento não é a data de coleta dos dados financeiros ou legislativos. Confira período e procedência na fonte correspondente.
- O catálogo é curado; ele não representa uma descoberta automática de tudo o que o Senado publica.
- A causa exata da negociação TLS permanece sem diagnóstico conclusivo. O contorno deve ser reavaliado com novas evidências antes de ser removido.
- A disponibilidade atual de cada endpoint exige uma verificação própria. Abrir o portal não comprova o funcionamento de todos os downloads e consultas.

O objetivo é deixar um caminho verificável: quem consulta entende de onde veio o dado; quem desenvolve consegue localizar a decisão técnica e testar seus limites.

Referência técnica: o clone traz a versão atual do repositório; esta análise descreve o commit abaixo.

Código analisado: [36f34316a6041ab0bb09e343b545bf6669b99ad5](https://github.com/legisbrasil/legisbrasil/commit/36f34316a6041ab0bb09e343b545bf6669b99ad5)